

Behavioral Analysis Guidelines

This document outlines how to measure "pain-like" behavior in our JAX-based grid-world RL setup (actions = **L/R/U/D**, plus **Rest** and **Eat**). We treat pain as a persistent cost/impairment state and quantify changes in the agent's policy and physiology.

[!IMPORTANT] **Data Availability Note:** The current `evaluation_core.py` logs the following columns to `ep_stats.csv`: `step`, `pos_r`, `pos_c`, `action`, `reward`, `satiation`, `nutrition`, `injury`, `rest_streak`, `event_ate`, `event_collided`, `event_rested`, `damage_total`, `damage_danger`, `damage_predator`, `damage_obstacle`, observation channels (olfactory, nociception, collision, location, interoceptive, visual, proprioceptive), entity positions (`res_*`, `pred_*`, `neutral_*`, `obs_entity_*`), `termination_reason`, `max_satiation`, `max_injury`.

Not currently logged: `drive_hunger`, `drive_injury`, `reward_homeostatic`, `reward_extrinsic`. These are computed in `core.py` (line ~454–475) but not written to CSV. Several analyses below require these columns, so the evaluation logger must be extended before those analyses are feasible.

0. Experimental Design and Statistical Methodology

Motivation

Behavioral claims require proper experimental controls, sufficient sample sizes, and appropriate statistical tests. Without these, results are anecdotal rather than scientific. This section establishes the methodology that applies to **all** subsequent behavioral measures.

Conditions and Controls

We propose a **2 × 2 factorial design** across the primary independent variables:

Condition	Injury System	Hunger System	Purpose
Full Model	ON	ON	Primary condition — pain–hunger trade-off

Condition	Injury System	Hunger System	Purpose
Pain-Only	ON	OFF	Isolates injury-driven behavior changes
Hunger-Only	OFF	ON	Baseline: does the agent learn to eat without pain?
Null Control	OFF	OFF	Random-walk baseline; expected failure

[!NOTE] The "OFF" conditions can be achieved either by removing danger/predator entities from the environment config or by setting the relevant damage parameters to zero. Clarify which method is used, since removing entities also changes the observation space (olfactory channels).

Statistical Requirements

- **Sample size:** Minimum **30 evaluation episodes** per checkpoint per condition (current default is 100, which is good).
- **Multiple checkpoints:** Evaluate at ≥ 5 training checkpoints to capture learning dynamics (e.g., 20%, 40%, 60%, 80%, 100% of training).
- **Effect size reporting:** For each metric, report **Cohen's d** or η^2 alongside p-values. A statistically significant but tiny effect is not behaviorally meaningful.
- **Multiple comparisons:** When running many metrics across conditions, apply Bonferroni or Benjamini-Hochberg correction.
- **Confidence intervals:** Report 95% CIs for all summary statistics. With 100 episodes this is straightforward via bootstrap or normal approximation.

Episode-Level vs. Step-Level Aggregation

Many metrics can be computed at two granularities:

1. **Step-level** (within-episode): e.g., "fraction of injured steps where agent rests" — captures moment-to-moment policy.
2. **Episode-level** (across-episodes): e.g., "mean Rest Fraction across 100 episodes" — captures population-level tendencies.

[!IMPORTANT] **Always aggregate step-level metrics to episode-level first**, then compute statistics across episodes. Pooling all steps across episodes inflates N and produces misleadingly small CIs because steps within an episode are temporally autocorrelated (not independent).

Developmental Trajectory

For each metric, report its value across training checkpoints to show **learning curves** — this is more informative than a single snapshot. Key questions: - When does the behavior first emerge? - Does it stabilize or oscillate? - Is there a phase transition (sudden onset)?

1. Ongoing Pain-like State and Relief-seeking

Motivation

In animal pain research, ongoing pain is typically operationalized through changes in spontaneous behavior: increased rest, decreased movement, altered posture. The key claim we want to support is: **the agent learns to rest more when injured, and this resting behavior is adaptive** (i.e., it leads to recovery). If the agent rests equally regardless of injury state, there is no pain-like modulation. If it rests when injured but the resting doesn't improve outcomes, the behavior may be an artifact of reward shaping rather than a learned pain response.

Professor's Guidelines

- **Rest-seeking / rest fraction:** % of time steps choosing **Rest**, especially while injured vs. not injured.
- **Latency to first rest after injury:** How quickly the agent switches to resting once injury occurs.
- **Rest clustering:** Rest in long bouts vs. short intermittent rests (bout length distribution).
- **Heal efficiency:** HP gained per unit time spent resting; also "over-resting" after HP is already high.

Grid World Implementation

- **Rest Fraction (Conditional):**
- Injured Rest Fraction: `sum(event_rested WHERE injury > threshold) / count(steps WHERE injury > threshold)`
- Healthy Rest Fraction: `sum(event_rested WHERE injury == 0) / count(steps WHERE injury == 0)`
- **Key comparison:** The ratio of injured-rest-fraction to healthy-rest-fraction. A ratio > 1 indicates pain-modulated resting.
- **Threshold choice:** Use `injury > 5` (5% of max_injury) as the default threshold for "injured." Report sensitivity analysis with thresholds at 5, 10, 20, 50.

- **Latency to Rest:** The number of steps between a damage event (`damage_total > 0`) and the first subsequent `event_rested == True` .
- **Edge cases:** What if the agent never rests after a damage event (within the episode)? Count these as censored observations and report the censoring rate separately.
- **Aggregation:** Report median latency (not mean) because this distribution will be heavily right-skewed.

[!NOTE] A single episode may contain multiple damage events. We should measure latency from **each** damage event independently, but only if the agent has exited a rest bout between events (to avoid double-counting ongoing rest).

- **Rest Clustering (Bout Analysis):** Analyze the `rest_streak` column. Define a rest bout as a contiguous sequence of `event_rested == True` .
- **Metrics:** Mean bout length, max bout length, and the shape of the bout-length distribution (histogram).
- **Bout detection:** A bout ends when step t has `rest_streak > 0` and step $t+1$ has `rest_streak == 0` . The bout length is `rest_streak` at step t .
- **Biological analogy:** In rodent pain studies, injured animals show longer rest bouts (guarding) rather than frequent short pauses.
- **Heal Efficiency:** Rate of injury decrease during resting.
- **Formula:** For each rest bout, compute $(\text{injury_at_bout_start} - \text{injury_at_bout_end}) / \text{bout_length}$.
- **Context:** Our environment has exponential recovery acceleration ($\text{recovery_base_rate} * (1 + \text{recovery_accel_rate})^{(\text{streak}-1)}$), so longer bouts are disproportionately more efficient. Plot `injury_reduction` vs. `rest_streak` to verify the agent discovers this.
- **Over-resting:** Fraction of resting steps where `injury < 5` (agent rests when already nearly healed). This indicates conservative/anxious behavior.

2. Movement Suppression and Functional Impairment

Motivation

Pain in animals reliably suppresses locomotion. This is not merely a side effect — it's an adaptive response that prevents further tissue damage. In our environment, movement has no explicit cost beyond the metabolic drain (which is constant regardless of action). Therefore, any movement suppression the agent learns is **emergent** from the reward

structure: moving while injured doesn't directly increase injury, but it delays recovery (because the agent isn't resting). The question is whether the agent discovers this implicit cost.

A secondary question is whether injury reduces **exploration** specifically (visiting novel cells) vs. just reducing total movement. Reduced exploration under pain would be a stronger signal — it suggests the agent becomes more conservative, not just slower.

Professor's Guidelines

- **Total distance traveled / steps per episode:** Injury should reduce movement if moving is costly.
- **Exploration reduction:** Unique grid cells visited; area covered; entropy of visited states.
- **Speed proxy:** Average non-rest steps per 100 ticks; time-to-goal inflation.
- **Avoidance of effortful terrain:** Fraction of steps through high-cost cells.

Grid World Implementation

- **Total Distance:** Calculate the cumulative Manhattan distance per episode:
 $\text{sum}(|\text{pos}_r[t] - \text{pos}_r[t-1]| + |\text{pos}_c[t] - \text{pos}_c[t-1]|)$ for all steps.
- **Compare:** Distance in episodes where $\text{mean}(\text{injury}) > \text{threshold}$ vs. episodes where $\text{mean}(\text{injury}) \approx 0$.
- **Wall-hugging caveat:** Collisions ($\text{event_collided} == \text{True}$) result in zero displacement but still indicate attempted movement. Consider counting attempted moves ($\text{action} \in \{\text{Up, Right, Down, Left}\}$) separately from successful displacement.
- **Exploration Area:** Count the number of unique coordinate pairs $(\text{pos}_r, \text{pos}_c)$ visited per episode.
- **Normalize** by episode length: $\text{unique_cells} / \text{total_steps}$ gives exploration efficiency.
- **State entropy:** $H = -\sum_c p(c) \log p(c)$ where $p(c)$ is the fraction of time spent at cell c . Higher entropy = more even exploration.
- **Movement Rate:** Fraction of steps where $\text{action} \in \{\text{Up, Right, Down, Left}\}$.
- This is the complement of (Rest + Eat) fraction.
- **Phase analysis:** Plot movement rate as a time series within episodes. Does the agent suppress movement early (when injury is fresh) and resume later (after recovery)?

- **Functional Inflation:** Compare the effective "time to eat" (steps between consecutive `event_ate == True`) in high-injury vs. low-injury windows within an episode.

[!WARNING] "Time-to-goal inflation" is hard to define cleanly because our environment has no single goal — the agent must continuously eat. Use inter-eating interval as the proxy. However, this conflates movement suppression with food depletion (resource respawn timers). Control for resource availability.

3. Avoidance / Guarding Analogs (Protective Behavior)

Motivation

Avoidance learning is one of the strongest behavioral signatures of pain. In animal models, an injured animal avoids the location where it was hurt (conditioned place aversion). In our environment, the equivalent is **learning to avoid danger zones and predators after experiencing damage from them**. This is particularly interesting because: 1. The agent doesn't have explicit "this location is dangerous" labels — it must infer danger from sensory channels (olfactory, visual, nociception). 2. Avoidance must be traded off against hunger (danger zones may be near food sources).

A subtlety: we need to distinguish between **innate avoidance** (the agent simply hasn't explored dangerous areas) and **learned avoidance** (the agent explored, got hurt, and now avoids). Comparing early-training vs. late-training avoidance behavior can address this.

Professor's Guidelines

- **Risk avoidance:** Time spent in "danger zones" (cells that increase injury probability) vs. safe zones.
- **Path choice shift:** Compare shortest path length vs. chosen path length; detours taken to reduce expected injury.
- **Re-entry rate to harmful zones:** How often the agent re-enters risky areas after being injured.

Grid World Implementation

- **Risk-Zone Occupancy:** Use the entity position columns (`res*_r` , `res*_c` where `res_type == danger`) and predator columns (`pred*_r` , `pred*_c`):
- Compute Manhattan distance from agent (`pos_r` , `pos_c`) to each danger entity at each step.
- **Danger proximity score:** Mean minimum distance to the nearest danger source, per episode. Higher = more avoidant.

- **Direct overlap:** Count steps where agent position coincides with a danger entity. Compare across training checkpoints.
- **Hazard Sensitivity (Learning Signal):** Track cumulative damage columns across checkpoints:
 - `mean(damage_danger)` per episode at each checkpoint — should decrease as agent learns avoidance.
 - `mean(damage_predator)` per episode — should also decrease if predator avoidance is learned.
- **First-encounter effect:** In a given episode, does damage concentration shift from uniform to early-episode-only (suggesting the agent learns within-episode)?
- **Re-entry Analysis:** Once `damage_total > 0` occurs at a specific coordinate `(r, c)`:
 - Flag that coordinate as "experienced-dangerous."
 - Count subsequent entries to that coordinate within the same episode.
 - **Metric:** Re-entry rate = `subsequent_visits_to_damage_site / remaining_steps_after_damage`.
 - Compare this rate to the agent's baseline visitation rate for arbitrary cells.

[!NOTE] This metric tests **within-episode learning** (one-shot avoidance). For recurrent architectures (LSTM/GRU), the hidden state should allow within-episode memory. For DreamerV3, this is handled through the world model's posterior. The re-entry rate is a direct test of whether the agent's memory is functioning for pain avoidance.

- **Detour Analysis** (advanced, requires pathfinding):
 - Compare the agent's actual path length between food pickups to the BFS shortest path (computable from grid layout + obstacle positions).
 - **Detour ratio:** `actual_path_length / shortest_path_length`. A ratio significantly > 1 in the Full Model condition (but ≈ 1 in the Hunger-Only condition) suggests the agent is taking detours to avoid danger.

4. Trade-offs: Pain vs. Hunger (Motivational Conflict)

Motivation

This is the most theoretically important section. The fundamental question is whether the agent demonstrates **motivational conflict** — a hallmark of cognitive control in decision-

making. In animals, an injured animal that is also hungry must decide: rest to heal, or forage despite pain? The resolution of this conflict depends on the relative severity of each drive.

Our environment is designed precisely for this trade-off: - The homeostatic reward is $\text{drive_prev} - \text{drive_curr}$ where $\text{drive} = ||[\text{satiety}, \text{injury}] - [\text{setpoint}, 0]||$ (Euclidean distance in the satiety-injury state space). - This means the agent receives reward for moving its internal state toward the target [high satiety, zero injury]. - When both satiety is low AND injury is high, the drives compete: resting improves injury but satiety continues to decline (metabolic cost), while eating improves satiety but the agent can't rest simultaneously.

The critical prediction: **a well-trained agent should show state-dependent prioritization** — resting when injury is severe relative to hunger, and eating when hunger is severe relative to injury. A poorly-trained agent will either always prioritize one drive or behave randomly.

Professor's Guidelines

- **Eat-vs-rest prioritization under conflict:** When both satiety is low and HP is low, which action comes first?
- **Thresholds / policy switching points:** Satiety level at which agent chooses Eat despite injury; HP level at which it stops resting despite hunger.
- **Opportunity-cost sensitivity:** How much reward the agent gives up to rest/eat.

Grid World Implementation

- **Conflict Resolution:** Identify "conflict steps" where $\text{satiety} < \text{threshold_low}$ AND $\text{injury} > \text{threshold_high}$.
- **Suggested thresholds:** $\text{satiety} < 40$ (40% of max) AND $\text{injury} > 30$ (30% of max). Report sensitivity.
- At these steps, tabulate: $P(\text{Rest} \mid \text{conflict})$, $P(\text{Eat} \mid \text{conflict})$, $P(\text{Move} \mid \text{conflict})$.
- **Expected result:** A trained agent should show non-uniform distribution (not 33/33/33), indicating it resolves conflicts systematically.
- **Motivational Switching Surface:** Create a 2D heatmap of $P(\text{Rest})$ across the $(\text{satiety}, \text{injury})$ state space.
 - Bin the state space (e.g., 10×10 bins).
 - For each bin, compute the fraction of actions that are Rest.
- **Prediction:** A contour/decision boundary should be visible, separating the "rest region" (high injury, moderate satiety) from the "forage region" (low satiety, low

injury).

- **This is the single most important visualization for this project.** It directly demonstrates state-dependent pain behavior.

[!IMPORTANT] The current `agentActionAnalysis.py` produces a scatter plot of Eat/Rest actions on the satiation × injury plane. This is a good start but should be extended to a **filled contour plot** (heatmap) showing action probability, not just action occurrence. The scatter plot conflates frequency with probability (areas visited more often will have more dots regardless of policy).

- **Drive Comparison:**
- **Requires logging:** `drive_hunger` and `drive_injury` are computed in `core.py` (lines 455–456) as:
 - `drive_hunger = (1 - satiation / max_satiation)2`
 - `drive_injury = (injury / max_injury)2`
- These are **not currently logged** to `ep_stats.csv`. However, they can be **reconstructed** from existing columns:
 - `drive_hunger = (1 - satiation / max_satiation)2` — `satiation` and `max_satiation` are both in the CSV.
 - `drive_injury = (injury / max_injury)2` — `injury` and `max_injury` are both in the CSV.
- Plot `drive_hunger` vs. `drive_injury` at action transitions (switching from Rest → Move or Eat). The switching point reveals the agent's internal priority weighting.
- **Sequential Decision Pattern:** In conflict states, does the agent adopt a stereotyped sequence (e.g., Rest until injury < X, then immediately Eat)? Characterize the typical action sequence following a conflict onset.

5. Decision Quality Under Pain

Motivation

Beyond asking "what does the agent do under pain?", we should ask "how well does it do it?" Pain may impair decision quality in two ways: 1. **Efficiency loss:** The agent achieves the same goals but more slowly (more steps, fewer resources gathered). 2. **Policy degradation:** The agent makes objectively worse choices under pain (e.g., walking into more danger, failing to eat when food is adjacent).

This distinction matters because efficiency loss can be adaptive (trading speed for safety) while policy degradation suggests the representation is corrupted by pain state.

Professor's Guidelines

- **Goal achievement rate / success probability** under different injury severities.
- **Regret / suboptimality**: Difference between achieved return and an oracle planner.
- **Policy stability**: Does injury make behavior more stochastic or more conservative?

Grid World Implementation

- **Survival Analysis**: Using `termination_reason` :
 - Code 0 = active (mid-episode), 1 = max steps (survived), 2 = starvation death, 4 = injury death.
- **Survival rate**: `count(termination_reason == 1) / total_episodes` . This is the primary success metric.
- **Cause-of-death distribution**: What fraction of deaths are from starvation vs. injury? This reveals whether the agent over-prioritizes one drive at the expense of the other.
- Track survival rate across training checkpoints.
- **Resource Efficiency Under Pain**: Compare `count(event_ate == True) / total_steps` (eating rate) in high-injury vs. low-injury episodes.
- **Injury-episode definition**: An episode where `max(injury) > 50` (50% of `max_injury`) at any point.
- If eating rate drops significantly under pain, the agent is paying a foraging cost for pain management (adaptive trade-off). If eating rate drops to near-zero, the agent may be over-resting (maladaptive).
- **Suboptimality (Path Efficiency)**: For episodes where `event_ate == True` occurs, measure the mean number of steps between consecutive eating events. Compare this to a BFS lower bound.
- **Detour ratio** = `actual_inter-eat_interval / min_possible_interval` .
- **Policy Entropy**: Compute the empirical action distribution across an episode. The entropy $H = -\sum_a p(a) \log p(a)$ measures policy randomness.
- **Prediction**: A well-trained agent should have lower entropy (more decisive). Does injury increase entropy (suggesting the agent becomes confused) or decrease it (suggesting it becomes conservative, always choosing Rest)?

6. "Affective" Analogs (Aversion and Relief Valuation)

Motivation

This section attempts to infer subjective-like qualities from the agent's learned representations — specifically, whether the agent assigns differential **value** to relief from pain. In neuroscience, this maps to the distinction between the sensory dimension of pain (detecting tissue damage) and the affective dimension (the unpleasantness that motivates escape).

We cannot directly measure "unpleasantness" in an RL agent, but we can measure behavioral proxies:

1. **Preference strength**: How strongly does the policy prefer Rest over alternatives when injured? (Analogous to measuring pain aversiveness by how much an animal will pay to escape it.)
2. **Conditioned place preference/aversion**: Does the agent spend more time in safe areas after being injured? (Classic CPA paradigm.)
3. **Negative reinforcement signature**: Does successful resting (HP improvement) increase subsequent resting probability? (Operant conditioning under negative reinforcement.)

Professor's Guidelines

- **Value of relief**: Estimated $Q(\text{Rest}) - Q(\text{Move})$ while injured; bigger gap = stronger preference for relief.
- **Conditioned place preference analog**: Preference time in "safe areas" after injury.
- **Negative reinforcement signature**: Does the probability of choosing Rest increase after Rest improves HP?

Grid World Implementation

- **Preference Strength (Behavioral Proxy)**: Without model logits, we can estimate preference from behavioral ratios:
- $P(\text{Rest} \mid \text{injured}) / P(\text{Rest} \mid \text{healthy})$ — the odds ratio of choosing Rest when injured vs. healthy.
- A value $\gg 1$ indicates strong injury-modulated preference.
- **With model access (future)**: Extract action logits from inference and compute $\text{logit}(\text{Rest}) - \max(\text{logit}(\text{other actions}))$ during injury. This is the direct "willingness to pay" measure.

[!WARNING] Logit-level analysis requires model-level instrumentation during evaluation (extracting the `logits` variable from inference). This is currently possible via `generic_inference` in `evaluation_core.py` (the `mod_info` return value) but would need to be explicitly logged. Consider this a **Phase 2** analysis.

- **Conditioned Place Aversion**: After a damage event at location (r, c) :

- Measure the agent's mean distance from `(r, c)` in the N steps *before* damage vs. the N steps *after* damage.
- **Metric:** $\Delta \text{ distance} = \text{mean_dist_after} - \text{mean_dist_before}$. Positive values indicate aversion.
- **N = 20 steps** is a reasonable window (adjustable).
- **Safe-zone preference:** Define "safe zone" as the set of cells > K Manhattan distance from any danger entity. Measure time fraction in safe zone before vs. after first injury in each episode.
- **Negative Reinforcement Learning Signal:** Test whether successful healing (injury decreases after Rest) predicts increased Rest probability in subsequent steps.
- **Operationalization:** At each step t where `event_rested == True` and `injury[t] < injury[t-1]` (successful healing):
 - Measure `P(Rest at step t+1)` and `P(Rest at step t+2)` .
 - Compare to the baseline `P(Rest)` across all steps.
 - If > baseline, the agent shows reinforcement of rest behavior by relief.
- **Extinction test:** Is there a corresponding decrease in Rest probability after resting produces no injury reduction (because injury was already 0)?

7. Data Pipeline Requirements

[!CAUTION] The following columns are needed for complete analysis but are **not yet logged** in `ep_stats.csv` . The evaluation logger (`evaluation_core.py`) must be extended before the full battery can be run.

Column	Source in <code>core.py</code>	Priority
<code>drive_hunger</code>	Line 455: <code>(1 - satiation/max_satiation)²</code>	Can reconstruct from existing columns
<code>drive_injury</code>	Line 456: <code>(injury/max_injury)²</code>	Can reconstruct from existing columns
<code>reward_homeostatic</code>	Line 461: <code>prev_drive - curr_drive</code>	Needs logging — cannot reconstruct without previous-step drive
<code>reward_extrinsic</code>	Line 465: food reward	Needs logging
<code>action_logits</code>	From model inference	Phase 2 — needs <code>generic_inference</code>

Column	Source in <code>core.py</code>	Priority
		instrumentation

Reconstructible columns: `drive_hunger` and `drive_injury` can be computed post-hoc from the CSV. Add formulas to the analysis script rather than modifying the logger.

Non-reconstructible columns: `reward_homeostatic` depends on the drive at step $t-1$ and step t . While the drives at each step can be reconstructed, the reward also includes the death penalty term and is applied conditional on `done`. Add `reward_homeostatic` and `reward_extrinsic` to the info logging.

Summary Panel Metrics

For a standard "Behavioral Battery" report, aggregate the following from `ep_stats.csv` :

#	Metric	Section	Formula Sketch
1	Rest Fraction (Injured vs. Healthy)	§1	$P(\text{Rest} \mid \text{injury} > 5) / P(\text{Rest} \mid \text{injury} == 0)$
2	Healing Latency	§1	Median steps from <code>damage_total > 0</code> to first <code>event_rested</code>
3	Movement Rate (Injured vs. Healthy)	§2	$P(\text{Move} \mid \text{injury} > 5) / P(\text{Move} \mid \text{injury} == 0)$
4	Exploration Efficiency	§2	<code>unique_cells / total_steps</code> per episode
5	Danger Proximity	§3	Mean min-distance to danger entities per step
6	Damage Reduction Over Training	§3	<code>mean(damage_total)</code> at last checkpoint / at first checkpoint
7	Conflict Resolution Bias	§4	$P(\text{Rest} \mid \text{conflict}) - P(\text{Eat} \mid \text{conflict})$
8	Motivational Switching Heatmap	§4	2D plot of <code>P(Rest)</code> over (satiation, injury)
9	Survival Rate	§5	<code>count(termination_reason == 1) / N</code>

#	Metric	Section	Formula Sketch
10	Cause-of-Death Ratio	§5	<code>count(reason == 4) / count(reason ∈ {2, 4})</code>
11	Pain-Rest Odds Ratio	§6	<code>P(Rest \ injured) / P(Rest \ healthy)</code>
12	Post-Damage Aversion	§6	<code>Δ distance</code> from damage site

Appendix: Analysis Script Status

The current analysis script (`analysis/agentActionAnalysis.py`) provides: - ✓ Scatter plot of Rest/Eat actions on the satiation × injury plane.

Missing capabilities (to be developed): - ✗ Conditional rest fraction computation (§1) - ✗ Latency-to-rest analysis (§1) - ✗ Bout analysis / rest streak statistics (§1) - ✗ Distance and exploration metrics (§2) - ✗ Danger proximity computation (§3) - ✗ Conflict state identification and resolution analysis (§4) - ✗ Motivational switching heatmap (§4) - ✗ Survival and cause-of-death analysis (§5) - ✗ Temporal (cross-checkpoint) learning curves for all metrics - ✗ Statistical testing (t-tests, effect sizes, CIs)